



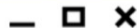
SECON 2021

19–20 июня



XII Международная конференция
разработчиков программного обеспечения

Тема лекции.txt



Использование JSON в Postgres

Иван Панченко

Postgres Professional,
сооснователь.
Москва

2002 г. Хранение полиморфных данных...

EAV? Другие способы нагромождения таблиц?

2003 (Pg 8.2) – расширение Hstore

– тип данных для набора пар "ключ-значение" (строка-строка)

`'a=>1, b=>2'::hstore`

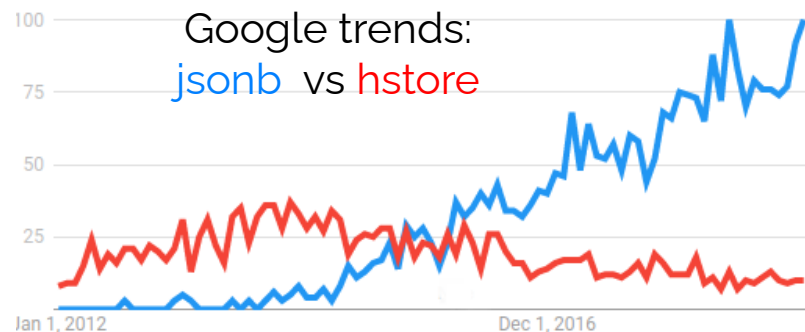
Операторы:

Извлечение: `hstore -> text`

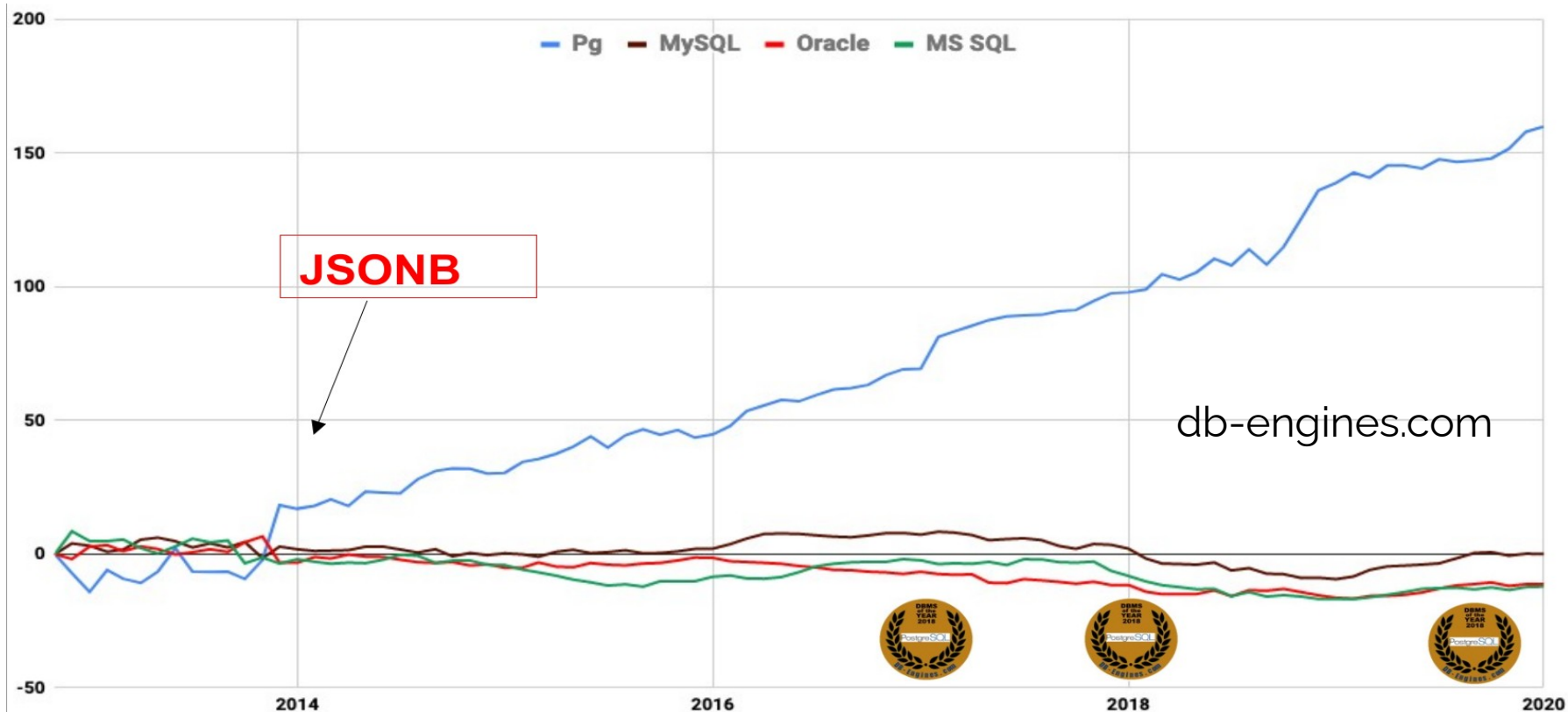
Есть ли ключ: `hstore ? text`

Contains: `hstore @ hstore`

Индексная поддержка GiST, GIN (с 2006)



JSONB: И тут попёрло...



JSON(B) спешит на помощь

JSON:

- Стандартный
- Многоуровневый
- JSONB (быстрый)

HStore:

- Быстрый
- Многоуровневый Hstore (proposal)

- 2011: Joseph Adams предложил JSON-расширение для 9.1 (не вошло)
<https://postgrespro.ru/list/id/BANLkTik1k6bdafEzi7xvnB8vy+hcv4iD1Q@mail.gmail.com>
- 2012: 9.2: тип данных json (хранение и верификация)
- 2012: Nested HStore proposal
- 2014: 9.4 JSONB (Бартунов, Коротков, Сигаев)
- 2016: Стандарт SQL:2016 поддержка JSON (Technical Report)
- 2019: 12 SQL/JSON (Бартунов, Коротков, Глухов)

<https://commitfest.postgresql.org/17/1471/>

В стандарт включена работа с JSON

4.46	JSON data handling in SQL.	174
4.46.1	Introduction.	174
4.46.2	Implied JSON data model.	175
4.46.3	SQL/JSON data model.	176
4.46.4	SQL/JSON functions.	177
4.46.5	Overview of SQL/JSON path language.	178
5	Lexical elements.	181
5.1	<SQL terminal character>.	181
5.2	<token> and <separator>.	185

Особенности SQL/JSON :2016

- Речь не идет о новом типе данных
- JSON текстовый
- Вводится понятие JSON Path, который чем-то похож на расширение JQuery, но шире:
 - Служит также для экстракции и фильтрации
 - Может быть мягким и жестким (lax и strict)
 - Есть способ передачи параметров из SQL-запроса в JSON Path

JSON или JSONB ?

JSON:

- Текстовый
- Порядок ключей сохраняется
- Дублирующиеся ключи
- Пробелы
- Занимает меньше места (м.б. даже вдвое, для коротких полей. Для длинных разница → 0)
- Быстрее вставка
- Нет оператора @> (contains)

JSONB:

- Быстрый при выборке по ключу (в 1000 раз!)
- Ключи не дублируются и отсортированы по длине и ключу
- Поддерживается индексирование

Вообще-то,
JSON не предназначался для
хранения!



1) Из текстового представления :

```
'{}'::jsonb
```

2) Функции-конструкторы:

```
jsonb_build_object(), array_to_json(), ...
```

<https://postgrespro.ru/docs/postgresql/13/functions-json>

3) Из объектов БД

```
SELECT row_to_json(pg_class) from pg_class;
```


Извлечение из JSON(B)

1) Операторы `json->text`, `json->>text`:

```
SELECT row_to_json(pg_class)->>'relname'  
FROM pg_class;
```

2) Операторы `json #> text[]`, `json #>>text[]`

3) Функции `json_each`, `json_each_text`, `json_array_elements`, `json_array_elements_text`

```
SELECT * FROM json_each_text('{ "a": "foo", "b": "bar" }')
```

key		value
a		foo
b		bar

4) Функция **jsonb_path_query_first (jsonb, jsonpath)**:

```
SELECT jsonb_path_query_first(  
    '{  
      "a":{  
        "x":  
          [2,3,4]  
      }  
}'::jsonb,  
    '$.a.x[1]'::jsonpath  
);
```

Какой будет результат?

4) Функция **jsonb_path_query_first (jsonb, jsonpath)**:

```
SELECT jsonb_path_query_first(  
    '{  
      "a":{  
        "x":  
          [2,3,4]  
      }  
}'::jsonb,  
    '$.a.x[1]'::jsonpath  
);
```

Какой будет результат? Правильно, **3**.

Поиск

1) Операторы `json->text`, `json->>text`:

WHERE `js ->>'key' = 'value'`;



2) Операторы `json #> text[]`, `json #>>text[]`

WHERE `js #>>ARRAY['key'] = 'value'`;



3) Функции `json_each`, `json_each_text` (SQL way)

WHERE EXISTS (
 SELECT * **FROM** `json_each_text(js)`
 WHERE `key='key' AND value='value'`
);

Поиск по "Contains"

```
{ "company_name": "Postgres Pro",  
  "offices": [  
    { "name": "Главный",  
      "city": "Москва",  
      "area": 1000  
    },  
    { "name": "Запасной",  
      "city": "Барнаул",  
      "area" : 200  
    }  
  ]  
}
```

Поиск по содержанию ("contains"). У кого офис в Барнауле?

WHERE js @> '{"offices":[{"city" : "Барнаул"}]}'

A light purple circle with a thin black border, containing the text "GIN" in bold black capital letters.

GIN

GIN-индекс по JSONB

```
CREATE INDEX idx ON tbl USING GIN (js);
```

```
Bitmap Heap Scan on tbl (cost=68.77..411.01 rows=100  
width=194)
```

```
  Recheck Cond: (js @> '{"offices":  
                      [{"city":"Барнаул"}]}'::jsonb)
```

```
    -> Bitmap Index Scan on idx (cost=0.00..68.75 rows=100  
width=0)
```

```
      Index Cond: (js @> '{"offices":  
                          [{"city":"Барнаул"}]}'::jsonb)
```



GIN

Поиск сложнее

У кого офис в Барнауле или Пензе?

```
WHERE js @> '{"offices": [ {"city" : "Барнаул"} ]}'  
OR  
      js @> '{"offices": [ {"city" : "Пенза"} ]}'
```

GIN

или

```
WHERE EXISTS (                                     -- SQL way  
  SELECT * FROM json_array_elements(js->'offices') t  
  WHERE t->>'city' IN ('Барнаул', 'Пенза')  
);
```

У кого офис в Барнауле или Пензе?

WHERE js @?



'\$.offices[*].city ? (@ == "Барнаул" || @=="Пенза")';

```
CREATE INDEX idx1 ON tbl  
  USING gin (js jsonb_path_ops);
```

Поиск ещё посложнее..

У кого **главный** офис в Барнауле или Пензе?

```
WHERE (  
  js @> '{"offices":[{"city" : "Барнаул"} ]}'  
OR  
  js @> '{"offices":[{"city" : "Пенза"} ]}'  
)  
AND  
  js @> '{"offices":[{"name" : "Главный"} ]}';
```

У кого **главный** офис в Барнауле или Пензе?

```
WHERE (  
  js @> '{"offices":[{"city" : "Барнаул"} ]}'  
  OR  
  js @> '{"offices":[{"city" : "Пенза"} ]}'  
  ) AND  
  js @> '{"offices":[{"name" : "Главный"} ]}';
```

```
WHERE (  
  js @> '{"offices":[{"  
    "city": "Барнаул",    "name": "Главный",  
  }]}'  
  OR  
  js @> '{"offices":[{"  
    "city": "Пенза",    "name": "Главный",  
  }]}';
```

То же, SQL way

У кого **главный** офис в Барнауле или Пензе?

```
WHERE EXISTS(  
  SELECT * FROM json_array_elements(js->'offices') t  
    WHERE t->>'city' IN ('Барнаул', 'Пенза')  
      AND t->>'name' = 'Главный'  
);
```

Ещё усложним запрос...

У кого **главный** офис в Барнауле или Пензе
с площадью **более 500 кв.м.?**

```
WHERE EXISTS(  
  SELECT * FROM json_array_elements(js->'offices') t  
    WHERE t->>'city' IN ('Барнаул', 'Пенза')  
      AND t->>'name' = 'Главный'  
      AND (t->>'area')::float > 500  -- для SQL – не проблема!  
);  
WHERE  
  js @?  
  '$.offices[*] ? (@.area > 500 && @.name == "Главный"  
    && (@.city == "Барнаул" || @.city == "Пенза"))'  
  ,  
  -- для JSONPATH – тоже!
```

Что ещё может SQL/JSON

Площадь главного офиса:

```
SELECT jsonb_path_query_first (js,  
    ' $.offices[*] ? (@.name == $x) .area',  
    vars=>'{"x":"Главный"}'  
)
```

Это PostgreSQL

```
FROM . . . ;
```

```
SELECT JSON_VALUE (js,  
    '$.offices[*] ? (@.name == $x) .area'  
    PASSING 'Главный' AS x  
    RETURNING int  
    DEFAULT -1 ON EMPTY  
    DEFAULT -2 ON ERROR) FROM . . .
```

Это стандарт

Антипаттерн №1

```
CREATE TABLE data (  
    fields jsonb  
);
```

```
CREATE TABLE data (  
    fields jsonb  
);  
CREATE INDEX data_pk  
ON data  
(fields->>'id');
```

```
CREATE TABLE data (  
    fields jsonb  
);  
  
CREATE INDEX data_pk  
    ON data  
    ((fields->'id'));  
  
CREATE INDEX data_pk  
    ON data  
    ((fields->'id')::int);
```

Какой из индексов лучше?

```
INSERT INTO foo
  SELECT jsonb_build_object('q', random(), 'id', x)
 FROM generate_series(100000000,200000000);
```

Думаете, сделали первичный ключ?



```
CREATE TABLE data_smth (  
    data_id int4 REFERENCES ..... ???? WTF  
);
```



```
ALTER TABLE data ADD CONSTRAINT  
foo_check CHECK (fields->>'id' IS  
NOT NULL);
```



Это вообще не первичный ключ.
Например, IDENTITY при логической репликации...

Издержки хранения всех полей в JSON(B)



Больше места (храним ключи + оверхед JSONB) не всегда!!



Извлекается дольше (особенно при TOAST)



index scan ~ такой же,
Seq scan 2-3 раза медленнее, даже без TOAST



Нет статистики. Поэтому планировщик будет ошибаться.



Нет всего разнообразия типов. Даже дат.



Меньше читаемость (иногда)

[Хорошая статья Dan Robinson](#)

Что такое TOAST ?

- В постгресе БД состоит из страниц размером 8k.
- Если запись больше страницы, значения полей прозрачным для пользователя образом бьются на слайсы по 2k, которые хранятся в отдельных спецтаблицах.
- Это называется TOAST.
- В нем есть ещё и компрессия.
- Чтобы извлечь поле из TOAST хотя бы частично, до Pg13 надо было достать всё целиком.
- Теперь – не целиком, а только до нужной позиции.
- Но и это – дорого.

Немного сократим издержки TOAST

```
CREATE TABLE data (  
    id  int4 primary key,  
    ..., -- некие поля, общие для всех записей  
    fast_small_fields jsonb, -- участвуют в  
селекции  
    large_slow_fields jsonb  
);
```

Какие поля есть смысл хранить в JSON(B) ?

Слабоструктурированные, т.е. :



Полиморфные*



Сильно разреженные*



Имеющие внутреннюю структуру (свойства файла, план запроса, ...)



Если надо хранить историю, несмотря на изменения в структуре*

* Но всё же стоит где-то хранить описание структуры.
И чем-то гарантировать отсутствие полного бардака.

Как экономить место с JSON ?

Объединить несколько записей в одну.

- Каких?
- Близких по смыслу, по времени, по пространственному расположению
- Например: часть временного ряда

Другая польза от JSON. Не связанная с хранением.

- Несколько запросов за раз.
- Агрегат сложной структуры
- Работа с рекурсивными структурами данных.

Агрегаты

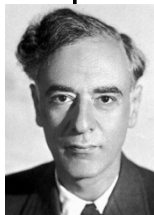
- Задача:
Получить список книг с авторами, по порядку



Теория поля.// Ландау Л.Д., Лифшиц Е.М.
Язык программирования С.// Керниган Б.В., Ритчи Д.М.

Или даже так

Теория поля.

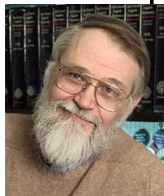


Ландау Л. Д.,



Лифшиц Е. М.

Язык программирования С.



Керниган Б. В.,



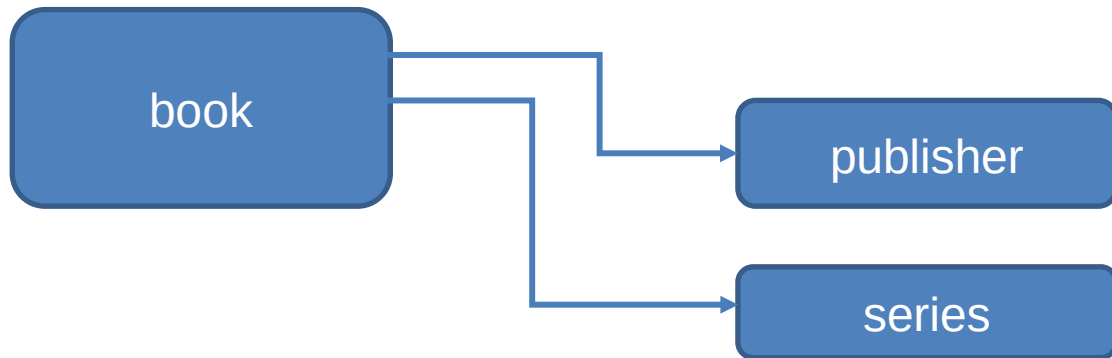
Ритчи Д. М.

JSON-агрегаты

```
SELECT book.*,  
  ( SELECT json_agg(row_to_json(x))  
    FROM (  
      SELECT person.*  
      FROM person  
      JOIN authorship ON person.id = person_id  
      WHERE book_id = book.id  
      ORDER BY pos  
    ) x  
  ) authors  
FROM book;
```

Другая польза от row_to_json

- Раскрытие атрибутов по ссылкам из внешних таблиц



```
SELECT book.*,  
        row_to_json(series)      AS $series,  
        row_to_json(publisher) AS $publisher  
FROM book  
LEFT JOIN publisher ON published.id = book.publisher  
LEFT JOIN series    ON series.id    = book.series
```


Начнем играть в рекурсию....

Задача:

Есть книжки и рубрикатор-дерево. Достать книги с полной рубрикацией.

```
CREATE TABLE sections (      -- Рубрикатор
  id      int PRIMARY KEY,
  parent  int REFERENCES sections(id),
  title   text
);
```

```
CREATE TABLE book (      -- Книги
```

```
  ...
  sections[] int
  ...
); { sections: [ /* Пути от корня до каждой рубрики книги */
  [ {id:..., title:...}, {id:...} ... ] -- 1я рубрика
  [ {id:...} ... ] -- 2я рубрика
  ...
]}
```

Хотим:

Тестовые данные для задачи

INSERT INTO section **VALUES**

(1, NULL, 'СУБД'),

(2, 1, 'PostgreSQL'),

(3, NULL, 'OC'),

(4, 3, 'Solaris');

INSERT INTO book **VALUES** (

'Postgres for Solaris OS',

ARRAY[2,4]

);



```
WITH RECURSIVE path AS (  
    SELECT s.id, s.title, s.parent , 0 AS level  
        FROM section s WHERE s.id = 4  
    UNION ALL  
    SELECT s.id, s.title, s.parent, path.level+1 AS  
level  
        FROM section s JOIN path ON s.id = path.parent  
)  
SELECT id,title FROM path ORDER BY level DESC;
```

id интересующего
нас узла

id		title
-----+		
3		OC
4		Solaris

(2 rows)

Шаг 2. Получим из него JSON

```
SELECT json_agg(row_to_json(path)) AS path
FROM (
```

```
    WITH RECURSIVE path AS (
        SELECT s.id, s.title, s.parent , 0 AS level
        FROM section s WHERE s.id = 4
        UNION ALL
        SELECT s.id, s.title, s.parent, path.level+1 AS level
        FROM section s JOIN path ON s.id = path.parent
    )
    SELECT id,title FROM path ORDER BY level DESC
```

```
) path;
```

path

```
[{"id":3,"title":"OC"}, {"id":4,"title":"Solaris"}]
(1 row)
```

Шаг 3. Получим несколько путей за один раз

```
SELECT * FROM (  
    SELECT unnest(ARRAY[2,4]) node_id  
) nodes  
JOIN LATERAL (  
    SELECT json_agg(row_to_json(path)) AS path
```

```
    FROM (  
        WITH RECURSIVE path AS (  
            SELECT s.id, s.title, s.parent, 0 AS level  
            FROM section s WHERE s.id = node_id  
            UNION ALL  
            SELECT s.id, s.title, s.parent, path.level+1 AS level  
            FROM section s JOIN path ON s.id = path.parent  
        )  
        SELECT id,title FROM path ORDER BY level DESC  
    ) path  
) AS path ON true;
```

node_id	path
---------	------

2	[{"id":1,"title":"СУБД"}, {"id":2,"title":"PostgreSQL"}]
4	[{"id":3,"title":"ОС"}, {"id":4,"title":"Solaris"}]

(2 rows)

Шаг 4. Итог.

```
SELECT title, (  
  SELECT json_agg(path.path)  
  FROM (  
    SELECT unnest(book.sections) node_id  
  ) nodes  
  JOIN LATERAL (  
    SELECT json_agg(row_to_json(path)) path  
    FROM (  
      WITH RECURSIVE path AS (  
        SELECT s.id, s.title, s.parent , 0 AS level  
        FROM section s WHERE s.id = node_id  
        UNION ALL  
        SELECT s.id, s.title, s.parent, path.level+1 AS level  
        FROM section s JOIN path ON s.id = path.parent  
      )  
      SELECT id,title FROM path ORDER BY level DESC  
    ) path  
  ) path ON true  
) paths FROM book;
```

Результат ,который мы хотели

```
-[ RECORD 1 ]-----  
title | Postgres for Solaris OS  
paths |  
[  
  [{"id":1,"title":"СУБД"}, {"id":3,"title":"PostgreSQL"}],  
  [{"id":2,"title":"ОС"}, {"id":4,"title":"Solaris"}]  
]
```

Disclaimer:

- Помните, что чем сложнее запрос, тем хуже оптимизатору

Доставайте только те
данные, которые вам
нужны!
Долой **SELECT *** !

Расслабляемся. 2 запроса в одном.

Задача: выполнить несколько запросов и вернуть результат вместе.

SELECT

```
( SELECT json_agg(row_to_json(book)) FROM book)::jsonb  
||  
( SELECT json_agg(row_to_json(section)) FROM  
section)::jsonb;
```

Или

```
SELECT json_build_object(  
    'books',  
    ( SELECT json_agg(row_to_json(book)) FROM book ),  
    'sections',  
    ( SELECT json_agg(row_to_json(section)) FROM section ));
```


Ещё пример: Агрегат-гистограмма

- Одним агрегатом получить целую гистограмму
- Возьмем упрощенную модель. Найдем частоты появления различных случайных чисел от 0 до 10
- Данные для примера:

```
SELECT( random()*10)::int  
  FROM generate_series(1,1000);
```

Решение

```
CREATE OR REPLACE FUNCTION freq_agg_f (state jsonb, item int)
RETURNS jsonb LANGUAGE sql IMMUTABLE STRICT AS $$
    SELECT jsonb_set(
        state,
        ARRAY[item]::text[],
        to_jsonb(COALESCE((state->>(item::text))::int, 0) + 1),
        true
    );
$$;

CREATE AGGREGATE freq_agg (int) (
    SFUNC=freq_agg_f, STYPE=jsonb, INITCOND='{}'
);
```

```
SELECT freq_agg (  (random()*10)::int ) FROM  
generate_series(1,1000);
```

freq_agg

```
{ { "0": 43, "1": 103, "2": 107, "3": 90, "4": 96, "5": 110,  
    "6": 100, "7": 111, "8": 100, "9": 99, "10": 41 }
```

(1 row)

- SQL и PL/PgSQL с JSON работают громоздко, и по-видимому, неэффективно

А что PL/Perl, PL/Python, PL/v8 ?

- PL/Perl и PL/Python получают JSON текстом, если не использовать TRANSFORM
- PL/v8 получает JSON сразу во внутреннем представлении
- PL/v8 не входит в комплект PostgreSQL и обычно отстаёт по версиям

Сложный агрегат на PL/v8

Агрегирует сразу сумму и список значений

```
CREATE EXTENSION plv8;  
CREATE FUNCTION v8_agg_f (state jsonb, item int) RETURNS  
    jsonb LANGUAGE plv8 IMMUTABLE STRICT AS $$  
    state.list.push(item);  
    state.sum += item;  
    return state;  
$$;  
  
CREATE AGGREGATE v8_agg (int) (SFUNC=v8_agg_f,  
    STYPE=jsonb,  
    INITCOND='{"list":[], "sum": 0}');
```

Тот же агрегат на PL/Perl

```
CREATE EXTENSION plperl;
CREATE EXTENSION jsonb_plperl; -- определяет TRANSFORM
CREATE FUNCTION pl_agg_f (state jsonb, item int) RETURNS jsonb
    LANGUAGE plperl TRANSFORM FOR TYPE jsonb IMMUTABLE STRICT
    AS $$
    my ($state, $item) = @_;
    push @{$state->{list}}, $item;
    $state->{sum} += $item;
    return $state;
$$;

CREATE AGGREGATE pl_agg (int)( SFUNC=pl_agg_f, STYPE=jsonb,
    INITCOND='{"list":[], "sum": 0}');
SELECT pl_agg ( (random()*10)::int ) FROM
    generate_series(1,1000);
```

Тот же агрегат на SQL

```
CREATE FUNCTION sql_agg_f (state jsonb, item int) RETURNS
  jsonb
LANGUAGE sql IMMUTABLE STRICT AS $$
  SELECT jsonb_set(
    jsonb_set(
      state,
      ARRAY['list', '2000000000'],
      to_jsonb(item), true
    ),
    ARRAY['sum'],
    to_jsonb( (state->>'sum')::int + item ),
    true
  );
$;
```

Это трюк. Мы не можем здесь
сделать push, поэтому пишем
"далеко за хвост" массива

```
CREATE AGGREGATE sql_agg (int) ( SFUNC=sql_agg_f,
  STYPE=jsonb, INITCOND='{ "list": [], "sum": 0 }');
```

На закуску: засада с NULL

Функция `jsonb_set` имеет атрибут `STRICT`

```
SELECT jsonb_set (some_json, path, NULL)
```

Будет, неожиданно, `NULL` !

Надо так:

```
SELECT jsonb_set (some_json, path,  
                  'null'::jsonb)
```


На 2ю закуску: та же засада, вид сбоку

```
SELECT ('{"X":null}'::jsonb -> 'X') IS  
NULL;  
?column?  
-----  
f  
(1 строка)
```

null в JSON – не NULL в SQL !
Расслабьтесь и получите удовольствие.

```
SELECT ('{"X":null}'::jsonb ->> 'X') IS  
NULL;  
?column?  
-----  
t  
(1 строка)
```

Объяснение: попробуйте...

```
SELECT 'null'::jsonb IS NULL;
```

Перспективы

- JSON_TABLE (Часть SQL:2016)
- Единый тип данных для JSON (ожидается в стандарте SQL)
- Lazy Transform для PL/*
- Более быстрый поиск внутри
- Оптимизация TOAST

Спасибо за внимание!

SECON 2021

19–20 июня

XII Международная конференция
разработчиков программного обеспечения

Контакты спикера.txt

Иван Панченко

info@postgrespro.ru

<https://postgrespro.ru/>

<https://pgconf.ru/>

